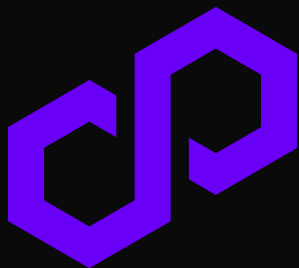




Formal Verification Report



Polygon sPOL Staking

April 2026

Prepared for Polygon

Table of Contents

Project Summary	4
Project Scope	4
Project Overview	4
Protocol Overview	4
Verification Methodology	5
Verification Notations	6
General Assumptions and Simplifications	6
Properties Overview	9
Threat and Security Overview	10
Security Overview	10
Design Overview	10
Threat Model	11
Assets	11
Actors	11
Trust Assumptions	11
Attack Vectors	12
Findings Summary	14
Severity Matrix	14
Detailed Findings	15
Informational Severity Issues	15
I-01 User may complete a buy without receiving anything in return	15
Properties in Detail	16
HLP_1. Correct Validator Management	16
List of Properties	16
Assumptions	16
Validator data structure integrity	16
Total staked balance equals the sum of individual validator stakes	18
Validator's status transition	18
Total balance equals sum of stakes	19
HLP_2. Valid State Conditions	20
List of Properties	20
Assumptions	20
Properties	20
Fee reserve never exceeds total delegation balance	20
Zero backing implies zero sPOL supply	20
	2

maxDivergence bounded	21
rewardFee never exceeds MAX_FEE	21
HLP_3. Integrity of important methods	22
List of Properties	22
Assumptions	22
Integrity of important methods	22
removeValidator postconditions	23
addValidator postconditions	24
restakeValidator postconditions	24
HLP_4. Correct Nonces Management	26
List of Properties	26
Assumptions	26
Nonce monotonicity and integrity	26
Withdrawal nonce uniqueness and FIFO consumption	27
Nonce truncation detection	27
HLP_5. Correct Fees Handling	28
List of Properties	28
Assumptions	28
feeReceiver preserved non-zero	28
takeFee no-op when zero fee balance	28
HLP_6. Exchange Rate Never Decreases	30
List of Properties	30
Assumptions	30
Exchange Rate Monotonicity	30
HLP_7. Funds conservation	32
List of Properties	32
Assumptions	32
Funds Conservation	32
Disclaimer	34
About Certora	34

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polygon sPOL Staking	https://github.com/OxPolygon/sPOL-contracts	3c4bdf6 (formal verification)	Solidity

Project Overview

This document describes the formal verification of **Polygon sPOL Staking**.

Timeline – March 11th to April 23rd, 2026

The following contract list is included in our scope:

OxPolygon/sPOL-contracts/tree/dev/src/sPOL.sol

OxPolygon/sPOL-contracts/tree/dev/src/sPOLController.sol

The team performed formal verification of the files in scope focusing on the L1 functionality. During the verification, the Certora team discovered bugs in the code, as listed on the respective section.

Protocol Overview

The sPOL protocol is a Liquid Staking Token (LST) system for the Polygon network, centered around the *sPOL* token which represents staked POL. Users deposit POL into the *sPOLController* on Layer 1 (Ethereum), where it is delegated to Polygon validators to earn staking rewards, while they receive *sPOL* tokens that appreciate in value as rewards accrue. The system includes a cross-chain component managed by *sPOLMessenger* and *sPOLChild*, enabling users on Layer 2 to mint and redeem sPOL seamlessly. The protocol manages liquidity through a "backfill" and "migration" mechanism, moving excess POL between layers to ensure withdrawals can always be

processed. It also supports converting existing validator shares (dPOL) directly into sPOL, providing a unified liquid staking interface.

Verification Methodology

We performed verification of the **Polygon sPOL** Staking protocol using the Certora Prover, a tool based on *Satisfiability Modulo Theories* (SMT). The Certora Prover works by compiling formal specifications written in the *Certora Verification Language* (CVL) together with the protocol's Solidity implementation into a mathematical model that is then checked exhaustively.

More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology, it means the CVL specification holds for all possible inputs and reachable states. However, specifications introduce assumptions to rule out situations that are impossible in realistic scenarios (e.g., to bound input ranges or exclude privileged administrative operations). Additionally, SMT-based verification is computationally difficult, so we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task containing assumptions, symbolic calls to the relevant functionality, and assertions that are checked on all resulting states. If all assertions hold under the given assumptions, the rule is verified.

Inductive Invariants: Inductive invariants are proved by induction on the structure of the smart contract. Constructors and initialization functions serve as the base case; all other externally callable functions that can modify storage serve as step cases. For the base case, we show that the property holds after initialization. For step cases, we assume a state where the invariant holds (induction hypothesis), symbolically execute the function under investigation, and prove that the invariant still holds afterward.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue.
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

We use mock contracts for representing other active contracts outside the scope, e.g., the *ValidatorShare* and *stakeManager*. These mocks are bound to concrete Certora Prover contract instances and replicate the essential behavior of the real contracts. The *MockValidatorShare* in particular, replicates the core interface of the real *ValidatorShare*, retaining only the state variables and functions necessary for the verification. For example, for restaking functions, claiming rewards is always bundled with an update of position: each function settles the outstanding reward before performing their respective balance adjustments; afterward reward is reset to zero. Key simplifications relative to the real contract include: the use of a single global reward value rather than per-user reward accounting, no allowance tracking, and no interaction with an external stake manager.

The verification models scenarios involving three *MockValidatorShare* instances, i.e., a *three-validator scene*. This scene keeps the state space tractable while modeling all possible states of a validator, e.g., arbitrary values of variables such as *STATUS* or *depositShare*, as well as all possible states of their combination, e.g., arbitrary combinations of *STATUS* and *depositShare* values for each validator.

We use a wrapper around `sPOLController`, called a *harness*, that allows us to provide the specification files with features necessary for the verification.

All rules require `safeAssumptions()` as a precondition. This helper bundles two groups of invariants that the Prover assumes at the start of every rule. These invariants have been formally verified so requiring them in rules is proven safe.

- **Basic accounting invariants** (`requireAllBasicInvariants()`):
 - The reward fee never exceeds its cap
 - The fee reserve never exceeds total delegation
 - Partial sums of sPOL balances are well-formed and monotone
 - The sum of all individual sPOL balances equals `totalSupply()`
 - Zero backing implies zero supply
 - `maxDivergence` never exceeds its cap.
- **Validator data-structure invariants** (`requireAllValidatorInvariants()`):
 - Total staked balance equals the sum of individual validator stakes
 - Active validators have active status and non-zero stake
 - The validator list and the `activeValidators` array contain no duplicates
 - Active validators form a subset of the full list
 - Deactivated validators have zero deposit share
 - Non-active validators have zero stake.
 - Deposit shares sum to zero (initial state) or one hundred
 - The three active-validator contracts are distinct and use the mock shares
 - The stored validator index matches the array position.

Within `safeAssumptions()`, both array lengths (`validatorList` and `activeValidators`) are bounded to 2^{100} to prevent arithmetic overflow in loop counters and internal variables that track aggregate values.

Loops are inherently difficult for formal verification. The Prover handles loops via an approximation called *loop unrolling*. Loops in the contract are replaced by multiple copies of their bodies. If the Prover detects that a loop has a constant number of iterations, this loop is unrolled as many times as the constant. (E.g. when copying the entire array.) If the Prover cannot easily determine the number of loop iterations, it will unroll it for a fixed number of times. The default number is 1, but it can be configured using a flag called `loop_iter`. Assuming, for instance, that `loop_iter = 2`, if a loop in a given function would cause the loop to run more than

twice, the Prover approximates the loop by two copies of its body. In this case, since some modeled scenarios involve an iteration over lists with up to three validators, the main verification configuration uses `loop_iter = 3`.

In several properties we exclude admin methods, e.g., `initialize()`. That is, we prove that an invariant holds after every method call except these admin methods, which are not supposed to be called arbitrarily during normal operation. These property-specific assumptions (function exclusions, case-splitting strategies, loop-bound tightening) are documented in the Assumptions section of each high-level property below.

Properties Overview

The following table lists the *High-level Properties* (HLP) that were covered.

Index	Title	Status
HLP_1	Correct validator management	Verified
HLP_2	Valid-state conditions	Verified
HLP_3	Integrity of important methods	Verified
HLP_4	Correct nonces management	Verified
HLP_5	Correct fees handling	Verified
HLP_6	Exchange rate never decreases	Verified
HLP_7	Funds conservation	Violated

Each HLP comprises several lower-level properties that are presented in [Properties in Detail](#).
Each such property comprises rules and invariants that are formally verified by the Prover.

Threat and Security Overview

A complete version of this section was first presented in the audit report. We include a shorter version here for completeness, and moreover we list relevant FV properties for each [Attack Vector](#).

Security Overview

The sPOL protocol's safety depends on correct cross-chain state synchronization, the integrity of the L1-calculated exchange rate, and proper interaction with the Polygon StakeManager across multiple validator delegations. We focused on properties around exchange rate monotonicity, i.e., the dPOL to sPOL ratio must strictly increase or stay flat.

Design Overview

The *sPOLController* (L1) acts as the source of truth, routing POL deposits into a curated list of Polygon validators based on target shares and a *maxDivergence* capacity limit. It handles all reward restaking, protocol fee minting (*rewardFee*), and unbonding execution. The *sPOLChild* (L2) acts as a high-speed storefront, processing instant POL/sPOL swaps using a cached exchange rate pushed from L1. To protect the protocol from arbitrage during the lag between L1 yield accrual and L2 rate syncs, *sPOLChild* applies a *safetyFee* on deposits and enforces a *maxExchangeRateUpdateDelay* circuit breaker. Because L2 liquidity can become imbalanced, the design introduces an asynchronous Backfill and Migration state machine. Surplus L2 POL is "migrated" to L1 for staking, while L2 sPOL burns trigger "backfills" where L1 unbonds stake and bridges POL back to L2. This creates significant configuration and operational complexity: particularly around the *PolBridger*, which is required to physically burn native POL on L2 (via MRC20) and submit Merkle proofs on L1 because native POL lacks a standard bridge *withdrawFor* function. Lastly, the design correctly isolates risk by delegating execution authority to an *AccessManager*, but relies heavily on off-chain keepers to push exchange rates and complete backfills, making liveness highly dependent on off-chain operational discipline.

Threat Model

Assets

- **Native POL and MATIC Tokens:** The underlying backing assets locked in the Polygon *StakeManager* or held in the L1/L2 protocol buffers.
- **sPOL Token:** The liquid staking derivative minted on L1 and L2 representing the user's fractional claim on the *dPOL* pool.
- **Protocol Fee Revenue:** The *rewardFee* (up to 10%) taken from the liquid rewards, and the *safetyFee* (up to 1%) used to compensate for the L2 exchange rate update lag.
- **Exchange Rate Data:** The $\text{totaldPOLBalance} / \text{totalsPOLBalance}$ ratio dictated by L1 and cached on L2, dictating the minting/burning curves.

Actors

- **Protocol Admin / AccessManager:** Governance authority configuring fees, *maxDivergence*, validator whitelists, bridge addresses, and executing rescue functions.
- **sPOLController (L1):** Autonomous manager routing POL to *StakeManager* contract, calculating the canonical exchange rate and buying validator shares from their corresponding *ValidatorShare* contract.
- **sPOLChild (L2) & sPOLMessenger (L1):** Cross-chain coordinators handling local mints/burns, state sync messaging, and bridging.
- **Keepers / Permissionless Callers:** External actors triggering *balanceWithL1()*, *updateL2ExchangeRate()*, *completeBackfill()*, and reward restaking.
- **End Users:** Stakers and unstakers utilizing the protocol on either chain.

Trust Assumptions

- **Admin / Keeper Operational Discipline:** Proper tuning of the L2 *safetyFee* and *maxExchangeRateUpdateDelay* to outpace L1 yield volatility. Timely keeper execution of exchange rate updates and backfill completions; failure to update halts L2 trading completely.

Calling `restakeAllActiveValidators` often to account for the liquid rewards. (this is permissionless but it's better for a trusted entity to manage it as well)

- **Bridge Liveness and Integrity:** The Polygon PoS bridge accurately relays State Sync messages and MRC20 exit proofs without censorship, message corruption, or extended delays.
- **StakeManager Correctness:** The Polygon staking contracts (`StakeManager`, `ValidatorShare`) accurately report balances, correctly compound liquid rewards, and strictly enforce unbonding delays without accounting flaws.

Attack Vectors

- **Cross-Chain Slippage DoS:** Natural L1 exchange rate appreciation (surpassing the 0.3% L2 safety fee) while a migration message travels cross-chain breaks strict slippage checks upon L1 arrival, freezing the migration pipeline and deadlocking L2 state.

Relevant FV Properties:

- Exchange rate never decreases
- Round-trip conversion never profits user

- **Dead Token Accumulation on Local Settlement:** Asymmetric L2 buys locally fail to properly burn the differential sPOL, leaving orphaned token supply permanently trapped within the controller.

Relevant FV Properties:

- Exchange rate never decreases (through related invariants)
- Supply only changes through mint/burn
- Zero backing implies zero sPOL supply

- **Backfilled Liquidity Loss via Zero Address:** An uninitialized child state variable defaults to `address(0)`, causing the L1 `DepositManager` to bridge backfilled POL into an unrecoverable black hole on L2, leading to severe L2 insolvency.

- **Total Supply Burn on Partial Backfill:** L2 `requestBackfill` incorrectly instructs L1 to burn the entire net circulating L2 supply debt (`bridgeMissingSPOL`) to satisfy a partial L2 withdrawal, completely destroying the cross-chain 1:1 token backing.

Relevant FV Properties:

- Exchange rate never decreases
 - Zero backing implies zero sPOL supply
 - Total balance equals sum of stakes
- **Permissionless Backfill Deadlock:** `withdrawPOL` allows any user to process matured withdrawals on behalf of the L1 messenger prior to `completeBackfill` execution. This consumes the withdrawal nonces, forcing `completeBackfill` to revert and permanently bricking the L2 withdrawal pipeline.

Relevant FV Properties:

- `withdrawPOL()` integrity and liveness
 - Nonce monotonicity and integrity
 - Withdrawal nonce consumption and uniqueness
- **Exchange Rate Manipulation via Double-Counting:** `_buySPOLWithDPOLMulti` omits the local `totalStaked` update for incoming validator shares. Subsequent `reloadAllActiveValidatorInfo` executions double-count these deposited shares, permanently inflating the exchange rate and stealing value from future withdrawers.

Relevant FV Properties:

- Exchange rate never decreases
- Total balance equals sum of stakes
- Staked balance matches validator contract
- `buySPOLWithDPOL` integrity

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	0	0
Informational	1	1	-
Total	1	1	0

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	User may complete a buy without receiving anything in return	Informational	Acknowledged

Informational Severity Issues

I-01 User may complete a buy without receiving anything in return

Files:

src/sPOLController.sol

Description:

While purchasing sPOL, certain buy paths may complete, although the amount paid is too small thereby leading to the minting of zero sPOL. Thus, the user may lose funds after the transaction without receiving anything in return.

Recommendations:

Revert when the amount sent to the user would be 0.

Customer Response:

Acknowledged.

Properties in Detail

This section presents each HLP in detail. Each section examines an HLP and comprises a list of properties under this HLP, the assumptions, as well as the rules/invariants that were verified in the context of the property (in tabular form).

HLP_1: Correct Validator Management

List of Properties

- Validator data structure integrity
- Total staked balance equals the sum of individual validator stakes
- Validator's status transition
- Total balance equals sum of stakes

Assumptions

The rule `nonActiveHasZeroStake` excludes admin recovery functions that can modify `totalStaked`. For performance reasons, the invariant `activeAmountEqualsTotalSupply` was verified for `buySPOLWithDPOL` and `buySPOLWithDPOLPermit` based on a scene with one active validator.

Validator data structure integrity			
Status: Verified			
Rule Name	Status	Description	Link to rule report
inactiveNotInValidatorList	Verified	Every entry in <code>validatorList</code> has <code>status != INACTIVE</code> .	rule report

activeValidatorsHaveActiveStatus	Verified	Every entry in activeValidators[] has ValidatorStatus.ACTIVE.	rule report
activeStatusThenIsInActiveValidators	Verified	If status == ACTIVE then the validator is tracked in isInActiveValidators.	rule report
listedValidatorsContractsAreDistinct	Verified	No two distinct validator IDs share the same validatorContract.	rule report
nonActiveHasZeroStake	Verified	Any validator not in activeValidators has totalStaked == 0.	rule report
validatorList_unique	Verified	Each validator ID appears at most once in validatorList.	rule report
activeValidatorsSubsetOfValidatorList	Verified	Every entry in activeValidators is also present in validatorList.	rule report
activeValidators_unique	Verified	Each validator ID appears at most once in activeValidators.	rule report
deactivatedHasZeroDepositShare	Verified	A deactivated validators have zero deposit share.	rule report
validatorIndexIsCorrect	Verified	validators[i].index == i for all validator IDs.	rule report
depositSharesSumIsZeroOrHundred	Verified	The sum of active validators' depositShare values is either 0 or 100.	rule report
activeValsLengthNoMoreThanVals	Verified	The active list is never larger than the full validator list.	rule report

Total staked balance equals the sum of individual validator stakes

Status: Verified

Rule Name	Status	Description	Link to rule report
activeAmountEqualsTotalSupply	Verified	<i>For each validator in activeValidators[], activeAmount == totalSupply().</i>	rule report 1 2 3 4 5 6
totalStakedMatchesBalanceOf	Verified	<i>For each validator in activeValidators[], validator.totalStaked == controller.validators[validator].balanceOf(controller)</i>	rule report 1 2 3 4 5 6

Validator's status transition

Status: Verified

Rule Name	Status	Description	Link to rule report
canChangeStatus	Verified	<i>Only addValidator and removeValidator can change validators[id].status; all other functions preserve it.</i>	rule report
validatorStateTransition	Verified	<i>Validator status follows a sound state machine: INACTIVE → {INACTIVE, ACTIVE}, ACTIVE → {ACTIVE, DEACTIVATED}, DEACTIVATED → {DEACTIVATED}.</i>	rule report

Total balance equals sum of stakes

Status: Verified

Rule Name	Status	Description	Link to rule report
totalBalancelSumOfStakes	Verified	<i>totaldPOLBalance equals sum of all validators' totalStaked</i>	rule report

HLP_2. Valid State Conditions

List of Properties

- Fee reserve never exceeds total delegation balance
- Zero backing implies zero sPOL supply
- maxDivergence bounded
- rewardFee never exceeds MAX_FEE

Assumptions

The rule `zeroBackingImpliesZeroSupply` excludes admin recovery functions (`migrateValidator()`, `reloadValidatorInfo()`) that can reduce `totaldPOLBalance` via external reads.

Properties

Fee reserve never exceeds total delegation balance			
Status: Verified			
Rule Name	Status	Description	Link to rule report
feeReserveNeverExceedsTotalDelegation	Verified	<code>feedPOLBalance() <= totaldPOLBalance()</code>	rule report

Zero backing implies zero sPOL supply			
Status: Verified			

Rule Name	Status	Description	Link to rule report
zeroBackingImpliesZeroSupply	Verified	<i>When totaldPOLBalance equals feedPOLBalance, sPOL totalSupply must be zero.</i>	rule report

maxDivergence bounded

Status: Verified

Rule Name	Status	Description	Link to rule report
maxDivergenceNeverExceedsMax	Verified	<i>maxDivergence never exceeds MAX_DIVERGENCE</i>	rule report

rewardFee never exceeds MAX_FEE

Status: Verified

Rule Name	Status	Description	Link to rule report
rewardFeeNeverExceedsMaxFee	Verified	<i>rewardFee never exceeds MAX_FEE</i>	rule report

HLP_3. Integrity of important methods

List of Properties

- Integrity of important methods
- removeValidator postconditions
- addValidator postconditions
- restakeValidator postconditions

Assumptions

The rules `buySPOLSingleIntegrity` and `sellSPOLSingleIntegrity` verify the single-validator variants of `buySPOL()` and `sellSPOL()`, respectively. Multi-validator distribution correctness is covered by `selectValidatorsBuyIntegrity` via the multi-validator `buySPOL()`.

The rule `buySPOLRevertsWhenOverfunded` models the `_maxDeposit() == 0` condition and verifies that any positive amount triggers a revert.

Integrity of important methods			
Status: Verified			
Rule Name	Status	Description	Link to rule report
<code>buySPOLSingleIntegrity</code>	Verified	After <code>buySPOL</code> , (a) <code>sPOL minted == convertPOLtoSPOL(amount)</code> at pre-tx rate, (b) user POL decreases by amount, (c) <code>totaldPOLBalance</code> increases by at least amount (liquid rewards add more).	rule report
<code>sellSPOLSingleIntegrity</code>	Verified	After <code>sellSPOL</code> , (a) <code>sPOL burned == amount</code> , (b) nonce POL amount == <code>convertSPOLtoPOL(amount)</code> at pre-tx rate. Truncation in (b) see <code>sellSPOLNonceTruncationSafety</code> rulehaha.	rule report

selectValidatorsBuyIntegrity	Verified	<i>_selectValidators distributes the full amount (buy path)</i>	rule report
buySPOLWithDPOLIntegrity	Verified	<i>After buySPOLWithDPOL, (a) sPOL minted == convertPOLtoSPOL(amount) at pre-tx rate, (b) totaldPOLBalance increases by at least amount.</i>	rule report
buySPOLRevertsWhenOverfunded	Verified	<i>Models the _maxDeposit == 0 condition and verifies the function reverts for any positive amount.</i>	rule report
withdrawPOLTransfersPOL	Verified	<i>On success, withdrawPOL must increase the user's POL balance.</i>	rule report
withdrawPOLLiveness	Verified	<i>Given a single matured nonce in queue with all guards satisfied, withdrawPOL does not revert.</i>	rule report
buySPOLRevertsForZeroShareValidator	Verified	<i>buy reverts for zero-share validator</i>	rule report

removeValidator postconditions

Status: Verified

Rule Name	Status	Description	Link to rule report
removeValidatorPostconditions	Verified	<i>After a successful removeValidator() call, the length of validatorList is unchanged, activeValidators length decreases by 1, status becomes DEACTIVATED, depositShare remains 0.</i>	rule report
removeValidatorRevertsWhenRewardsPending	Verified	<i>RemoveValidator reverts when there are pending liquid rewards.</i>	rule report

addValidator postconditions

Status: Verified

Rule Name	Status	Description	Link to rule report
addValidatorPostconditions	Verified	After successful addValidator(), validatorList and activeValidators each grow by 1, status becomes ACTIVE, totalStaked == 0, depositShare == 0.	rule report
addValidatorOnlyAcceptsInactive	Verified	addValidator() reverts if the validator's current status is not INACTIVE.	rule report
addValidatorRevertsWhenNotActiveInStakeManager	Verified	addValidator() reverts when the stakeManager does not recognize the ID.	rule report

restakeValidator postconditions

Status: Verified

Rule Name	Status	Description	Link to rule report
restakeValidatorPostconditions	Verified	After successful restakeValidator(v): totalStaked and totaldPOLBalance increase by the same restaked amount, status remains ACTIVE.	rule report

restakeValidatorIndependentOfOtherLock	Verified	<i>restakeValidator(v) only accesses validators[v].validatorContract; it does not iterate activeValidators or check lock status.</i>	rule report
--	----------	--	-----------------------------

HLP_4. Correct Nonces Management

List of Properties

- Nonce monotonicity and integrity
- Withdrawal nonce uniqueness and FIFO consumption
- Nonce truncation detection

Assumptions

The rule `withdrawPOLLiveness` models the following preconditions: there is a single matured nonce in queue, the transient reentrancy guard not entered, the contract is not paused, there is no payable value, the user is not the controller, the user address is non-zero, and the controller holds enough POL to cover the withdrawal.

Nonce monotonicity and integrity			
Status: Verified			
Rule Name	Status	Description	Link to rule report
noncesLessThanGlobalNonce	Verified	<i>Every user nonce index holds a value strictly less than <code>globalWithdrawNonce</code>.</i>	rule report
nonceZeroNeverAllocated	Verified	<i>After initialization, nonce 0 has amount == 0 and no function changes it.</i>	rule report
globalWithdrawNonceMonotonicallyIncreases	Verified	<i><code>globalWithdrawNonce</code> is monotonically non-decreasing; initialize sets it to 1.</i>	rule report

Withdrawal nonce uniqueness and FIFO consumption

Status: Verified

Rule Name	Status	Description	Link to rule report
noncesAreUnique	Verified	<i>Nonces are unique.</i>	rule report
withdrawPOLConsumesFront	Verified	<i>withdrawPOL consumes the front nonce.</i>	rule report
withdrawTwiceReverts	Verified	<i>Double-withdraw on the same nonce reverts.</i>	rule report
allDequeuesWellFormed	Verified	<i>Double-ended Queues do not wrap around.</i>	rule report

Nonce truncation detection

Status: Verified

Rule Name	Status	Description	Link to rule report
sellSPOLNonceTruncationSafety	Verified	<i>Nonce amount matches expected POL (truncation detector)</i>	rule report

HLP_5. Correct Fees Handling

List of Properties

- feeReceiver preserved non-zero
- takeFee no-op when zero fee balance

Assumptions

The rule feeReceiverPreservedNonZero requires *feeReceiver()* != 0 as a precondition (once initialized via *initialize()* or *changeFeeReceiver()*, then it is always non-zero).

feeReceiver preserved non-zero

Status: Verified

Rule Name	Status	Description	Link to rule report
feeReceiverPreservedNonZero	Verified	<i>initialize and changeFeeReceiver both require non-zero. No other function writes feeReceiver. Once non-zero, always non-zero.</i>	rule report

takeFee no-op when zero fee balance

Status: Verified

Rule Name	Status	Description	Link to rule report

takeFeeNoOpWhenZeroFeeBalance	Verified	<i>_takeFee returns early when currentFeedPOLBalance == 0.</i>	rule report
takeFeeNoOpWhenZeroFeeBalance_stronger	Verified	<i>When feedPOLBalance == 0 before takeFee, the entire storage is unchanged.</i>	rule report

HLP_6. Exchange Rate Never Decreases

List of Properties

- Exchange rate monotonicity

Assumptions

For the rules `exchangeRateMonotonicity_simple` and `exchangeRateMonotonicity_multi`, `_applyPermit()` is replaced by a function that produces non-deterministically an arbitrary value; this is sound because `_applyPermit()` only writes permit nonces and does not affect the accounting state. For performance reasons, this rule is split into two separate configurations: one for single-validator operations and another for multi-validator buy/sell paths that use `_selectValidators()`. Four supporting inductive invariants on partial sums of sPOL balances establish that `totalSupply()` equals the full sum of all individual balances, which enables the Prover to model valid states.

Exchange Rate Monotonicity			
Status: Verified			
Rule Name	Status	Description	Link to rule report
partialSumsOfBalances Monotone	Verified	<i>Partial sums of balances are monotonic.</i>	rule report
partialSumsOfBalancesStartAtZero	Verified	<i>Partial sums of balances start at 0.</i>	rule report
partialSumsOfBalancesGrowsCorrectly	Verified	<i>Each partial sum increases by exactly the corresponding sPOL balance.</i>	rule report

sumOfSPOLBalancesEqualsTotalSupply	Verified	Sum of User sPOL balances equals totalSupply.	rule report
exchangeRateMonotonicity_simple	Verified	Exchange rate never decreases after non-multi-validator functions.	rule report
exchangeRateMonotonicity_multi	Verified	Exchange rate never decreases after multi-validator functions.	rule report 1 2 3 4

HLP_7. Funds conservation

List of Properties

- Funds conservation

Assumptions

The rules `polDecreasedIffSpolReceived` and `polDecreasedIffSpolReceived_amountBound` exclude `buySPOLWithDPOL()` and `buySPOLWithDPOLPermit()`: the user sends validator shares and not POL in those paths). The part of the rule that verifies buy paths requires that the deposit mints at least one sPOL.

The rules `dpolDecreasedIffSpolReceived` and `dpolDecreasedIffSpolReceived` exclude POL buy paths, i.e., `buySPOL()` and `buySPOLPermit()` variant: the user sends POL and not dPOL in these paths. The dPOL balance is approximated as the sum of validator share balances across the three mock validator contracts.

All four rules above exclude admin functions.

Funds Conservation			
Status: Violated			
Rule Name	Status	Description	Link to rule report
roundingFavoursProtocol	Verified	<i>convertSPOLtoPOL(convertPOLtoSPOL(x)) <= x by double floor-division.</i>	rule report
polDecreasedIffSpolReceived	Violated Reported issue I-01	<i>For every non-admin function, a user's POL balance decreases if and only if their sPOL balance increases.</i>	rule report

polDecreasedIffSpolReceived_amountBound	Verified	As polDecreasedIffSpolReceived, with the addition of the requirement that amount mints at least 1 sPOL	rule report
dpolDecreasedIffSpolReceived	Violated Reported issue I-Q1	For every non-admin function, a user's dPOL balance (sum of validator share balances) decreases iff their sPOL balance increases.	rule report
dpolDecreasedIffSpolReceived_amountBound	Verified	As dpolDecreasedIffSpolReceived, with the addition of the requirement that amount mints at least 1 sPOL	rule report

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.